ML Mini-lesson M1

What a neural network actually is, built up from first principles -- no training required to understand this, just the structure that Ollama runs when serving Qwen.

Piece 1 — A neuron = a tunable formula

DONE

Key ideas

- A single neuron computes: multiply each input by its own tunable number (a weight), sum the results, add one more tunable number (a bias). That's the entire computation -- multiply, add.
- 'Training' (not something you do yourself) is the separate process of nudging these tunable numbers, repeatedly, against real example data, until predictions get close to correct answers.
- Once training is done, 'running' the model is just plugging input through this fixed arithmetic -- no further learning happens at that point.

Core mechanism

```
output = (input1 * weight1) + (input2 * weight2) + ... + bias

Example: price = (size * 200) + (age * -500) + 10000
          200, -500, 10000 are the tunable numbers (weights / bias)
```

Piece 2 — A layer = many neurons run in parallel

DONE

Key ideas

- A layer runs several of the Piece-1 formulas side by side, on the exact same inputs, each with its OWN separate set of tunable numbers.
- Instead of producing one output number, a layer produces a whole LIST of output numbers -- one per neuron in that layer.
- Mechanically this is one matrix multiplication: inputVector x weightMatrix + biasVector.

Core mechanism

```
Layer input:  [size, age]
  neuron 1 -> A = (size*200)  + (age*-500) + 10000
  neuron 2 -> B = (size*-30)  + (age*15)   + 200
  neuron 3 -> C = (size*5)    + (age*300)  + -1000
Layer output: [A, B, C]
```

Piece 3 — Stacking layers (depth)

DONE

Key ideas

- Feed one layer's output list as the next layer's input list. Each layer's whole job: take the previous layer's output, run it through a fresh batch of Piece-1 formulas, produce a new list.
- The LAST layer's output is the final answer -- a price, a classification, or for an LLM, the 50,000-number next-token probability vector from Lesson 2.2.
- Number of stacked layers = the network's DEPTH. Size of each layer's output list = its WIDTH.

Core mechanism

```
Layer 1:  [size, age]  ->  [A, B, C]
Layer 2:  [A, B, C]    ->  [D, E]
Layer 3:  [D, E]       ->  [final_answer]
```

Piece 4 — Non-linearity -- why depth actually matters

DONE

Key ideas

- Surprising fact: stacking layers that are PURELY multiply-and-add mathematically collapses into one single equivalent layer -- no matter how many you stack, you gain nothing (pure linear chains always simplify: D = 3*(2*size) = 6*size, i.e. one multiplication).
- Fix: after each layer's output, before passing it to the next layer, apply a small non-linear snap. Most common: ReLU -- 'if the number is negative, output 0; otherwise output it unchanged.' (The Tensorflow Playground demo used a different one called Tanh -- same role.)
- This non-linear step is what breaks the 'collapses into one layer' shortcut -- once some values get zeroed/clipped and others don't, the layers can no longer be merged into one simple formula. This is what lets depth (many stacked layers) add real representational power.

Core mechanism

```
ReLU(x):  if x < 0: return 0   else: return x

Applied after every layer, before the next layer uses the output.
```

Takeaway: Full picture: weights/bias (tunable numbers) -> parallel formulas (a layer) -> stacking layers (depth) -> a non-linear snap between every layer so depth isn't wasted. That is a neural network, structurally, start to finish -- nothing more exotic hides underneath. An LLM's 'predict next token' function (2.1) IS exactly this structure (using an architecture called the Transformer), with tokens going in as numbers and the final layer's output being the probability vector over the vocabulary.

Hands-on — Tensorflow Playground experiment

DONE

Key ideas

- 6 hidden layers, 8 neurons each, Tanh activation, on a circular dataset: reached near-zero test loss (~0.000) after ~524 training epochs -- the learned decision boundary closely traced the real circular cluster.
- Reduced to 1 hidden layer, 2 neurons, on a diagonal-checkerboard dataset: stuck at test loss 0.157 / training loss 0.183, STILL stuck even after 2,763 epochs -- the network could only draw a single straight diagonal line, structurally incapable of the needed curved/multi-region boundary, no matter how much longer it trained.
- Direct, observed proof of Piece 4: a shallow network is capped in what SHAPE of pattern it can ever represent, regardless of training time. More layers/neurons raise that structural ceiling, not just the training speed.

Takeaway: Connecting to language: each layer can only build on patterns the previous layer already detected (e.g. layer 1: 'this token is a noun' -> layer 2: 'this noun is the subject' -> ... -> layer 20: 'this pronoun refers to the person mentioned two paragraphs back'). You cannot skip straight to that final insight in one layer, exactly as the 1-layer network could not skip straight to a curved boundary. Real LLMs use dozens of layers because language's true complexity (grammar + meaning + context + reasoning) needs many rounds of that compounding refinement, not one or two.

This mini-lesson is conceptual background only -- no training performed, no code written. Context for everything Ollama/Qwen has been doing under the hood since Lesson 2.3.